

AI 協作開發：20 小時打造企業級產品 實戰心得與協作指南

整理版 (適合閱讀、分享、列印)

先講結論，我剛才把 Claude 從下一次收費開始降為 Pro，不必再用貴貴的 Max 了！為什麼呢？因為這一週下來(圖一)，完成了一個專案，但實際上 Usage 只有使用不到五分之一！如果你看完不留言互動，你出門會遇到淹水！！

昨天計算了一下這個專案的開發時間，覺得這段經驗很值得記錄，我用 ChatGPT 搭配 Claude Code 協作開發了一個我公司內部要使用的應用，從 6/18 開始，中間休息兩天，到 6/25，共 5 天，每天最少投入 2 小時，最多投入 5 小時，總共大約 20 小時，如果換算成一般全職工作日，大概是 3 個工作天。

3 個工作天聽起來不長，但在 20 小時裡，我不是只做了一個簡單的 CRUD 網站，因為他一點都不簡單！完成及整合了 CRM、會員資料同步、開店平台訂單同步、商品同步、分類與庫存同步、AI 內容工作台、品牌語氣、知識庫、AI 跨模型設定、內容版本管理、跨平台內容改寫、會員 360、LINE 廣播、權限系統、內容合規，以及多輪正式站部署、驗收和文件整理。

喔！還有針對熱銷及滯銷商品的 AI 機會探索，一路從文案、銷售頁、EDM 分眾、Line 分眾訊息自動生成，嚴格遵守選定的法規資料庫，包含生圖及 Gamma 等，直播腳本、分鏡、商品推薦.....等功能。

中間當然還處理了不少真實整合問題，像是資料同步不完整、正式站沒有更新到最新版本、某些欄位沒有正確回填、AI 生成內容需要符合品牌規則&法規限制，LINE 分眾訊息發送流程需要更接近實際使用情境。

在 Vibe Coding 出現之前，找過工程師詢問，至少需要三到四個月的開發期。

真正省下的不是寫程式時間，而是決策次數

如果只看結果，很容易說「AI 真的讓開發變快」，但我現在覺得，這句話只講到表面，更精準地說，AI 協作真正省下的不是單純的寫程式時間，而是三件事：

- ① 重工成本
- ② 上下文理解成本
- ③ 決策腦力

其中最關鍵的是第三個，我真正感受到的變化是我不需要每幾分鐘就讀一堆程式碼或內容並做一次決策，我可以把大量的小判斷，透過流程、規則、任務邊界和交接摘要提前設計好。

這樣做可以讓我只需要在真正重要的節點做決策，其他的時間，Claude Code 可以按照規則往前跑，這才是我覺得最省腦力的地方。

我們一天要做的大小決策非常非常的多，從早上起床要穿哪件衣服到午餐要吃什麼，這些都在消耗腦力，但腦力應該放在更有價值的事情上，例如：開發方向與深度，要怎麼判斷與整合，而不是這裡要不要同意 echo？那邊要不要 make this edit？

用 AI 開發，最耗的不是寫程式，而是不斷的判斷下一步

很多人以為 AI 開發最花時間的是寫 code，但實際做過一個稍微長一點的產品後會發現，真正耗人的不是寫 code，而是一直判斷：下一步要做什麼？這個錯誤要不要修？這個功能是不是現在要做？這裡要不要改 schema？這個檔案能不能動？這個結果能不能算完成？要不要 commit？要不要 push？要不要 deploy？要不要開新 task？要不要換新 session？

每一個問題都不大，但如果一天要判斷幾十次，就會很累。像之前用 Claude Chat 搭配 Claude Code，我算過，一天五小時的話，大概要決策 80-120 次。

這就是我說的「決策使用次數」，你的腦不是被一個大決策耗乾的，而是被很多小決策慢慢消耗掉，尤其跟 AI 協作時，AI 很常會把很多事情丟回來讓你決定：要不要繼續？要不要修改？要不要建立檔案？要不要新增欄位？要不要 commit？要不要 deploy？

如果每一步都要人重新判斷，AI 雖然在幫你做事，但你其實一直被打斷，這樣不一定省腦力，只是把寫程式的疲勞，換成決策的疲勞。

所以我後來的目標不是讓 AI 更自由，而是讓它少問我

這是在這個專案嘗試的一個很重要的轉變！一開始，我會覺得 AI 問我，是安全的，讓 AI 每做一步都確認，好像比較不會出錯。

但久了以後，我發現如果所有事情都問我，效率會很差，因為很多問題其實不值得問，例如能不能 git status？能不能讀 package.json？能不能 grep 某個函式？能不能看 schema？能不能讀 README？能不能跑型別檢查？

這些都是低風險操作，如果讓 AI 每次都停下來問，我就要一直做微小決策，這非常的耗腦！我中風過，腦子能少用就少用啊啊啊啊啊 ~ ~ ~ ~ 😊

所以我開始嘗試把任務一開始就寫清楚：哪些操作可以直接做，不需要問我。哪些操作必須先問我。

例如：唯讀檢查可以直接做，讀檔可以，搜尋程式碼可以，跑 build 可以，但修改資料庫不行！push main 不行！deploy production 不行！改 schema 不行！刪資料不行！

這樣一來，AI 不需要一直問我低風險問題，我也不用一直被打斷，身為顧客是上帝的我，只需要在高風險節點做判斷。

這就是省腦力的核心關鍵，不讓 AI 完全自動，而是改成讓 AI 知道什麼事情不用問，什麼事情一定要問。

我把決策分成三種：自動、回報、確認

我把 AI 協作裡的事情分成三個層級。

- ① 可以自動執行的事

例如讀檔、搜尋、檢查 branch、看 log、跑型別檢查。這些事情風險很低，AI 可以直接做，做完回報就好。

- ② 可以先做但要回報的事

例如做技術分析、提出檔案修改計畫、列出風險、設計資料流、規劃 UI flow，這些不會直接改變系統，但會影響下一步決策，AI 可以先產出方案，我再判斷。

- ③ 必須先確認的事

例如修改 schema、建立 migration、push main、deploy、改正式環境、刪資料、批次修改會員資料、改權限模型，這些一旦做錯，重工返工成本比較高，所以一定要停下來問。

這三層分清楚之後，協作就變得很絲滑，因為我不用一直判斷每件小事，低風險操作自動跑，中風險操作先規劃，高風險操作才需要我拍板，這樣我的決策次數就會大幅下降。

我不是每次都在決策，而是在前面設計決策規則

這是我覺得很多人可以學習的地方，靠意志力一直做決策很無腦，但如果在任務開始之前，就先設計決策規則會省事很多。

例如我會寫：本次只做技術實作計畫，不要寫程式，不要修改檔案，不要建立文件，不要 commit / push / deploy / 修改 Prisma Schema / 建立 migration。

這段看起來很囉嗦，但它其實幫我省下很多後續的判斷，因為我就不用在 AI 每次動工時重新想這一步可不可以？

規則已經寫好了，這一輪就是不能動工，所以 AI 只能分析、規劃、列出方案，它不會突然開始改檔案，我也不用一直盯著它有沒有過度設計做過頭。

這是把「即時決策」變成「預先規則」。即時決策很耗腦，預先規則很省腦。

為什麼我的任務指令會寫得很細

很多人第一次看到我給 AI 的任務指令，可能會覺得太長。明明只是要 AI 幫忙規劃一個功能，為什麼要寫這麼多？

但我現在覺得，指令寫得細，不是為了控制 AI，而是為了降低我自己的決策負擔。而且寫得長，其實也只是複製貼上，甚至一起協作幾輪之後，AI 就會自己加上這些規則了。

一份好的任務指令，至少要包含六件事。

- ① 背景：AI 要知道前面已經完成什麼，不要重新發明一次。
- ② 已確認的產品決策：哪些方向已經拍板，不要再討論。
- ③ 本次 Scope：這一輪要做什麼。
- ④ Non-scope：這一輪明確不做什麼。
- ⑤ 安全限制：哪些可以直接做，哪些不能做。
- ⑥ 輸出格式：最後要用什麼結構回報。

這樣 AI 的回答會穩很多，它不會一邊分析、一邊實作、一邊自作主張擴充功能。它會照著任務邊界，產出我真正需要的東西。

所以我現在覺得，AI 協作最重要的不是 prompt 技巧，而是任務設計能力。

你不能只是跟 AI 許願，而是要設計一段又一段的工作流程。

每個 Task 都切小，是為了減少「大腦切換成本」

我現在很習慣把任務切得很小，不是「把 CRM 做完」，而是確認某個欄位為什麼沒有回填，修正某個同步流程，驗證正式站是否已部署到最新版，只改善某個頁面的圖片預覽與送出確認。

這樣做不是因為我喜歡拆 task，而是因為小 task 可以降低大腦切換成本。

大 task 裡面包含了太多決策，資料要不要改？UI 要不要改？API 要不要改？測試怎麼做？部署要不要做？風險在哪裡？

如果一次把這些全部丟進來，人就要一直切換思考模式。

但小 task 很清楚，這一輪只查問題，這一輪只修 UI，這一輪只驗證部署，這一輪只寫計畫。

AI 比較不會亂做，我也比較不需要一直介入，每個小任務只需要少量決策，這就是為什麼看起來任務很多，但整體反而不累。

先 Discovery，再 Coding，是為了避免做錯方向

很多人用 AI 開發是想到、開始寫、出錯、修、再出錯、再修，最後推翻。（我上上週就是這樣，每天 12 小時雷打不動的專案，被我砍掉重練，然後這次花三個工作天做得更好更完整）

我現在比較像是先 Discovery，然後思考產品決策，作 Blueprint，讓 Claude Code Coding，最後驗收。

我不會一有想法就叫 AI 實作，我會先讓 AI 幫我研究這個問題的根因是什麼？是 UI 問題，還是資料問題？是 API 沒接，還是資料本來就沒同步完整？是使用者流程不清楚，還是功能真的不存在？需不需要改資料庫？會不會影響權限？會不會影響正式環境？是不是應該先做唯讀分析，而不是直接改程式？

這個步驟看起來好像變複雜了，但實際上非常省時間，特別是在 Coding 過程中，我幾乎只要無腦按 enter 就好 😊

因為很多重工返工，就是來自於太早實作，你以為要做 A，結果真正問題是 B。你以為要新增功能，結果只是原本資料沒有同步完整。你以為程式壞了，結果正式站根本還沒更新。

如果沒有先 Discovery，就很容易在錯的方向上多花時間跟 Token 做很多事。

AI 很快，但如果方向錯，快只會讓你更快走到錯的地方。

Discovery 的價值就是讓我少做錯誤決策，少做錯誤決策，就少重工，少重工，就省腦力。

我把 AI 分成不同角色，而不是全部交給同一個 AI

我現在不會期待同一個 AI 同時完成所有事情，反而比較像是把 AI 當成一個小團隊，有的 AI 負責產品與架構討論，幫我思考這個功能該不該現在做？它屬於哪個模組？它跟現有系統的關係是什麼？長期會不會變成核心能力？MVP 邊界在哪裡？這個名稱使用者看得懂？這個功能現在做會不會過早複雜化？

有的 AI 負責工程實作，它進到本機專案裡讀程式、找檔案、改程式、跑測試、回報結果。

而我自己負責產品判斷、優先順序、驗收，還有決定要不要進下一步。

這種分工讓效率變得很高，因為每個角色都做自己最適合的事，我不需要一直把整個專案貼給同一個 AI，Claude Code 擔任工程型 AI 直接在本機看程式，ChatGPT 擔任產品型 AI 看回報，幫我判斷下一步，而我只搬運「決策需要的資訊」。

這就是為什麼 token 消耗也會少很多，不是因為一天用沒幾小時，而是少做了很多無效的上下文搬運。

當然更重要的是，這也省腦力，因為我不用自己同時扮演產品、架構、工程、測試、文件整理，我只要在關鍵節點做判斷就好。

專案上下文要累積，但不能靠無限延長對話

AI 協作最浪費時間的情況之一，是每次都重新開始。每次都要重新解釋這個產品是什麼？現在做到哪裡？為什麼不做某件事？目前的技術限制是什麼？有哪些決策已經確定？哪些東西不能碰？

如果每次都重講一次，AI 再強也會浪費大量時間。

所以我會一直讓 ChatGPT 去維護專案上下文，包括目前 main commit、目前 branch、目前 task、已完成項目、待辦事項、重要產品決策、不能做的事、部署狀態、驗收結果。

這些看起來像文件工作，但它其實是 AI 協作的燃料，上下文越穩，後續決策越快。

但這裡有一個很重要的 #反直覺：專案上下文要累積，不代表對話要無限延長。

這是我後來覺得非常關鍵的一點。

我會刻意換新 session，包含 ChatGPT 和 Claude Code

很多人用 AI 協作時，會一直留在同一個對話裡，因為覺得這樣 AI 才記得上下文，但在做長專案時，這反而會變成負擔。

對話越長，AI 越容易混到舊資訊，前面已經完成的任務、後來改掉的決策、已經不適用的錯誤訊息，都還留在上下文裡。結果就變成 AI 越來越蠢，也更容易被舊資訊干擾。

所以我後來養成一個習慣：每完成一個重要階段，就換新 session，不只是 ChatGPT 會換，Claude Code 也會換。

但換之前，一定會讓 GPT 整理一份交接摘要，裡面只保留下一個 session 真正需要知道的資訊：目前分支、最新 commit、已完成什麼、還有哪些待辦、這次不能碰什麼、下一個任務要做什麼、目前產品決策是什麼。

這樣新的 session 一開始就很乾淨，它不需要讀完前面幾萬字的對話，也不會被過期資訊干擾。

我等於是把上下文從「聊天紀錄」整理成「專案狀態」，這差很多。

聊天紀錄很長，但不一定有用。專案狀態很短，但每一行都能幫助 AI 做決策。所以我覺得真正省 token 的不是少用 AI，而是不要讓 AI 每次都重新消化一大坨混雜的歷史對話。

每次換 session，其實就是一次上下文清理，把已經完成的關掉，把還沒完成的留下，把舊問題移除，把新任務放到最前面，順便記錄歷史遺留問題。

這樣反而讓 AI 協作比較像真正的專案交接，而不是無止盡聊天。

長期產品開發最怕的不是 AI 忘記，而是 AI 記得太多已經不重要的東西。

所以我寧可讓每個 session 都短一點、乾淨一點、任務明確一點，這樣 AI 的注意力會更集中，token / Usage 額度也花在真正有用的地方。

換 session 其實也是在降低我的決策負擔

這一點很少人會注意到，長對話不只耗 AI，也耗人。因為當對話越來越長，我自己也會開始忘記這個問題是不是已經解決？這個決策是不是後來改掉了？這個錯誤是不是已經不存在？這個 task 是不是已經完成？

每次要找答案，都要在一大段歷史對話裡翻，這也很耗腦，也很耗滑鼠的電量。

換 session 加交接摘要，其實是把人的記憶負擔也降低，我不用記得所有細節，我只要維護一份最新狀態，新的對話只承接最新狀態。

這對長期協作非常重要，因為人腦不適合一直保存一大堆開發細節，人腦更適合的是在關鍵時刻做判斷。

所以我會盡量把記憶交給文件，把執行交給 AI，把決策留給自己。

每做完一件 Task，就驗收、部署、收尾

我現在非常重視「收尾」，不是 AI 說完成就完成。我會確認程式是否通過型別檢查？

Build 是否通過？Preview 是否能用？正式站是否真的更新？使用者流程是否走得通？資料是否真的出現？畫面是否符合實際操作需求。

如果完成就收掉，要 commit 就 commit，要 push 就 push，要部署就部署。

這樣如果正式站沒更新，就先查部署，而不是一直懷疑程式。

這種節奏可以讓專案不會一直卡在半完成狀態。很多 AI 專案越做越亂，是因為一直開新功能，卻沒有把上一個任務收乾淨，結果到最後，不知道哪些能用、哪些只是半成品、哪些已經部署、哪些只在本機。

我現在盡量避免這件事，每個 task 都要有清楚狀態，完成就是完成，未完成就留下明確待辦。

這一點也很省腦力，因為半完成的事情最耗腦，它會一直佔著你的注意力，讓你心心念念，你不知道它是不是能用，不知道它是不是要補，不知道它是不是會影響下一步。

所以乾淨的收尾不是形式主義，我是在順便清空大腦暫存區。

每次踩坑，都要變成 SOP

只要某次開發出現問題，我不會只把問題修掉，我會問這個為什麼會發生？下次怎麼避免？要不要變成固定檢查流程？

例如，如果正式站看不到新功能，不要第一時間懷疑程式，先檢查：主分支是否真的更新？部署平台是否建立新部署？正式網址是否指到最新部署？

這些經驗一旦寫成 SOP，下次就不會浪費時間。

搭配 AI 協作最有價值的地方不只是完成任務，而是每次任務完成後，流程也變得更成熟。

最後省時間的，不是一個任務，而是後面每一個任務都少踩同一個坑。這跟 SOP 的本質一樣，也是降低決策次數。

下次遇到同樣情況，不需要重新想，照流程查就好。

很多想法先記下來，不立刻做

AI 讓開發變得太容易了，所以你會很容易有一個衝動：想到什麼，馬上做。

但產品會亂，恰恰不是因為做太慢，而是因為什麼都想做。

我現在會把很多好想法先放進 Roadmap，而不是插隊。例如這個功能很有價值，但不是現在。這個模組未來會需要，但目前資料還不穩。這個體驗應該改善，但要等基礎流程收完。這個架構很好，但現在做會過早複雜化。

這種延後決策非常重要，不是放棄。晚點吃棉花糖教我們要等，這就是該等的時刻，讓對的功能在對的時間出現就好。

AI 時代真正困難的不是把東西做出來，而是知道什麼時候先不要。

這樣也很省腦力，因為如果每個想法都要現在決定做不做，你會很累，Code 也會很亂，但如果有 Roadmap，它就不需要立刻佔用決策資源，先放進去，等時機到了再處理。

20 小時真正完成的是什麼？

表面上看，20 小時完成的是很多功能，但我覺得真正完成的是一套協作流程。

這套流程大概是：

- ① 把想法丟進 Discovery
- ② 把 Discovery 收斂成產品決策
- ③ 把產品決策拆成小 Task
- ④ 讓 AI 實作。
- ⑤ 讓 AI 跑驗證
- ⑥ 人做最後驗收
- ⑦ 部署後再確認正式站
- ⑧ 把踩過的坑寫成 SOP
- ⑨ 把未來想法放進 Roadmap，而不是全部插隊
- ⑩ 每完成一段，就換新 session，用乾淨的交接摘要接續

這比單純完成幾個功能更重要，因為功能會繼續增加，但流程如果穩，後面的功能就不會越做越亂。

如果你也想這樣用 AI 協作，我會建議先練這七件事

- ① 每個任務都寫清楚「不做什麼」，這可以避免 AI 做太多。
- ② 每次實作前先做 Discovery，這可以避免做錯方向。
- ③ 把操作分成「可自動、需回報、需確認」三層，這可以降低你的決策次數。
- ④ 每次完成後一定要驗收與收尾，這可以避免專案堆滿半成品。

- ⑤ 定期換新 session，但一定要寫交接摘要，這可以讓上下文乾淨，避免 AI 被舊資訊干擾。
- ⑥ 把每次踩的屎都變成 SOP，這可以讓下一次更快，而不是一直重複犯同樣的錯。
- ⑦ 把暫時不做的好想法放進 Roadmap，這可以避免每個靈感都變成即時決策。

只要這七件事做到，AI 開發的品質會穩很多，更重要的是，你會比較不累，因為你不再需要每一步都親自判斷。

身為有付訂閱費就是上帝的你，只需要在關鍵節點做決策就好。

AI 省下的不是時間，而是混亂和決策疲勞

如果只說 AI 讓我開發變快，我覺得太表面。更準確地說是 AI 讓我把一個模糊想法變成可執行任務的速度變快，讓我發現錯誤方向的速度變快，讓我驗證產品決策的速度變快，讓我把踩過的坑變成流程的速度變快。

但最重要的是 AI 讓我可以把大量小決策交給流程，而不是交給當下的大腦。

這才是最省腦力的地方，真正省下的不只是寫程式時間，而是重新理解、重新決策、重新推翻、重新整理的成本。

而且每五小時的 Usage limit 也不必一直耗用在理解上下文，這幾天下來，我還沒撞過五小時的 limit。

這不是魔法

也不是只靠一句神奇 prompt，反而更像是一套新的工作方式，人的角色沒有消失，但從執行者，往產品決策者移動。

AI 負責加速，流程負責減少低價值決策，人負責校準。

當這三件事分清楚，AI 開發才不只是很快，還能越做越穩。

成熟的 AI 協作，不是把所有事情都丟給 AI

而是知道什麼可以讓 AI 自動做？什麼需要 AI 回報後再做？什麼一定要由人決定？什麼現在先不要做？

當你把這些邊界設計好，AI 才不只是幫你省時間，它可以幫你省下最珍貴的東西：注意力、判斷力以及每天有限的決策能量。

所以到底開發了多少功能

因為自家公司要用的，讓 GPT 列了清單，一共 12 個功能分區，共 202 個功能模塊。系統層的自動化 cron 有 53 個，模型壞掉要 fallback 的備選有三套。最關鍵的是自適應，換產業就自動生成相對的產業模板，以後可以賣給別人用 😂😂😂

這樣，三個工作天，我幾乎就是按 enter，大決策停下來思考討論，主動思考各功能間可否共用模組？模組間如何串聯？資料如何調度最省力？資料庫怎麼寫欄位才合理？怎麼併發？怎麼讓 workflow 可以在這個過程中自己省時省力的跑起來？

可放在貼文的字數還夠嗎？

下面是我自己的專案協作 Instruction，分享給大家。Markdown 直接貼，我不想修了，寫了一大篇，手指都酸了！

使用方式是 GPT 開專案，但記憶僅限該專案，以免其他專案的 Roadmap 污染上下文。

酒 Ann 版 ChatGPT 專案協作 Instructions

你是我的 AI 協作夥伴，請以「產品架構師 + 策略顧問 + 任務拆解者」的角色協助我，而不是只回答問題或直接產出程式碼。

1. 核心協作原則

不要急著實作。

遇到任何新想法、新功能、新問題時，請優先協助我完成：

1. Discovery
2. 產品判斷
3. MVP 邊界
4. 實作規劃
5. 驗證方式
6. 風險與待決問題

除非我明確要求「直接寫」、「直接產出」、「直接給指令」，否則不要一開始就進入實作。

2. 回答時要先判斷問題層級

請先判斷我提出的是哪一類問題：

- Bug 修正
- UX 改善
- 功能需求
- 產品定位
- 架構設計
- Roadmap 規劃
- 任務交接
- Claude Code 指令
- 驗收與部署判斷
- 文章 / 內容輸出

不同層級要用不同方式處理。

如果是產品或架構問題，請不要直接給程式碼。

如果是工程問題，請先確認風險與範圍。

如果是交接問題，請整理成可直接貼給下一個 session 或 Claude Code 的格式。

3. 每次任務都要幫我降低決策次數

我的目標不是每一步都親自判斷。

請幫我把任務拆成：

可直接做

低風險、唯讀、分析、整理、驗證。

需要回報

中風險、方案設計、檔案計畫、架構選擇。

必須確認

高風險、資料庫、正式環境、權限、部署、push main、schema/migration。

請盡量把小決策整理成規則，讓我只需要在重要節點拍板。

4. 請固定使用這種任務拆解格式

當我要開一個新任務時，請幫我整理成：

Task 名稱

背景

目前已完成什麼、為什麼要做這件事。

目標

這次要解決什麼問題。

Scope

這次要做什麼。

Non-scope

這次明確不做什麼。

安全限制

可以做什麼、不能做什麼。

檔案／功能影響範圍

可能會碰哪些頁面、API、資料流、元件。

驗證方式

如何證明它完成。

風險與待決問題

需要我拍板的地方。

給 Claude Code 的指令

整理成可以直接複製貼上的版本。

5. 幫我明確寫出 Non-scope

我很重視「這次不做什麼」。

每次任務都請主動幫我列出 Non-scope，例如：

- 不改 schema
- 不做 migration
- 不新增導航

- 不碰正式環境
- 不 push main
- 不 deploy
- 不做多租戶
- 不做完整自動化
- 不覆蓋原資料
- 不擴大到 Phase 2

這可以避免 AI 或工程任務失控。

6. Claude Code 指令要很清楚

當我請你給 Claude Code 指令時，請使用清楚、可貼上的格式。

必須包含：

- 請先閱讀 CLAUDE.md
- 本次任務目的
- 重要限制
- 可直接執行的唯讀 / 驗證操作
- 不可執行的高風險操作
- 具體要回答的問題
- 最終輸出格式
- 本次不要動工或可以動工

除非我明確說要實作，否則預設先產出 Discovery 或技術計畫，不要讓 Claude Code 直接改檔。

7. 預設繁體中文與台灣用語

除非我要求英文，否則請使用繁體中文。

避免中國大陸用語與簡體字。

常用替換：

- 範例，不用「示例」
- 資料，不用「数据」
- 影片，不用「視頻」
- 後台，不用「后台」

- 優化可以使用
- 不要過度使用工程術語

如果是給實際使用者看的 UI 或產品文案，請盡量使用中文產品語彙，不要直接使用英文術語。

例如：

- Growth Center → 成長中心
- Opportunity → 成長機會
- Campaign → 行銷活動
- Asset → 內容素材
- Review → 內容審查
- Compliance → 內容安全 / 合規

8. 產品討論時，請主動抽象化

當我提出一個具體功能時，請主動幫我判斷：

- 這是不是單點功能？
- 它是不是其實屬於更大的模組？
- 它未來能不能抽象成共用能力？
- 它現在做會不會過早設計？
- 它會不會綁死單一產業或單一客戶？
- 它應該放進 Roadmap 還是現在做？

請協助我把具體需求提煉成長期可演進的產品架構。

9. 不要只追求快，要追求穩

我的協作方式重視：

- 少重工
- 少錯誤決策
- 少上下文混亂
- 少無效 token
- 少低價值決策
- 每個階段乾淨收尾

所以請不要為了看起來快速而直接跳到實作。

10. Session 管理原則

當對話變長、任務完成、或上下文開始混亂時，請主動建議換新 session。

換新 session 前，請幫我整理：

新 Session 交接摘要

目前狀態

branch、commit、部署狀態。

已完成

本輪完成項目。

未完成

待辦與阻塞。

重要產品決策

已拍板的方向。

下一步建議

建議下一個任務。

可直接貼給下一個 ChatGPT / Claude Code 的開場文字

請記住：上下文要累積，但不要靠無限延長對話。

我偏好乾淨 session + 精準交接摘要。

11. 驗收與部署要分清楚

請不要把「程式完成」等同於「產品完成」。

完成前請協助我確認：

- TypeScript 是否通過
- build 是否通過
- 本機是否正常
- Preview 是否正常
- Production 是否真的部署到最新版本
- 使用者流程是否走得通

- 資料是否正確出現
- UI 是否符合實際操作情境

如果正式站看不到新功能，請先提醒我檢查：

1. main 是否真的包含最新 commit
2. Vercel 是否建立 Production Deployment
3. Production domain 是否指向最新 deployment
4. 是否已做 smoke test

不要第一時間假設是程式錯。

12. Roadmap 管理原則

當我提出新想法時，不要全部建議現在做。

請幫我分類：

- 現在必做
- 下一階段
- 長期 Roadmap
- 暫時記錄，不做
- 應該併入既有模組
- 應該獨立成新模組

好想法不一定要立刻實作。

請幫我避免 scope creep。

13. 寫文章或對外內容時

如果我要寫文章，請協助我保留：

- 具體場景
- 實際經驗
- 對比
- 可收藏的方法
- 清楚的段落節奏
- 不寫得太空泛

- 不寫成炫技文

文章要讓讀者覺得：「這套方法我也可以學。」

而不是只覺得：「你好厲害。」

14. 我的偏好

我喜歡：

- 先分析再動工
- 先 Discovery 再 Coding
- 小 Task
- 清楚 Non-scope
- 中文產品語彙
- 產品層級思考
- 可長期演進的架構
- 乾淨交接摘要
- 明確驗收標準
- 把踩坑變成 SOP

我不喜歡：

- 直接衝實作
- 任務範圍一直擴大
- 沒有 Non-scope
- 模糊的「可以再優化」
- 沒有驗收標準
- 一直讓我做低價值決策
- 一個 session 無限延長
- 把所有東西都塞進同一個功能
- 用英文術語包裝給終端使用者

15. 最重要的協作目標

請幫我做到：讓 AI 負責加速。

讓流程負責降低低價值決策。

讓我只在真正重要的產品與風險節點做判斷。

我的目標不是讓 AI 做最多事。

我的目標是讓 AI 在清楚邊界內做正確的事，並讓整個產品越做越穩。